Halal Stock Research Agent — Build Plan

Oct 27, 2026 · @afiq

Project overview

The project is **HalalLens**, a working name you can change. It is a multi-agent research assistant that takes a Bursa Malaysia stock and returns a cited research brief covering Shariah status, technicals and news. A human approves each brief before it is finalised. The build takes about 6 weeks at 8–10 hours a week, and it is research only: it never places trades.

User flow

1. The user types a request, for example "Analyse Maybank" or "Compare TENAGA and YTLPOWR".
2. A supervisor resolves the ticker and decides which specialist agents to call.
3. Three agents gather facts in parallel:
   - Shariah compliance
   - Technical analysis
   - News and sentiment
4. A writer agent drafts a brief with sources and a confidence note for each section.
5. The graph **pauses** for human review. The user can approve the brief, ask a follow-up, or ask the agents to dig deeper.
6. The final brief is saved, and the full run is traced in Langfuse.

In scope: Bursa Malaysia stocks, Shariah status, support and resistance levels, a candlestick summary, news sentiment, and a chat-style UI.

Out of scope: buy or sell signals, price prediction, order execution, and real-time tick data.

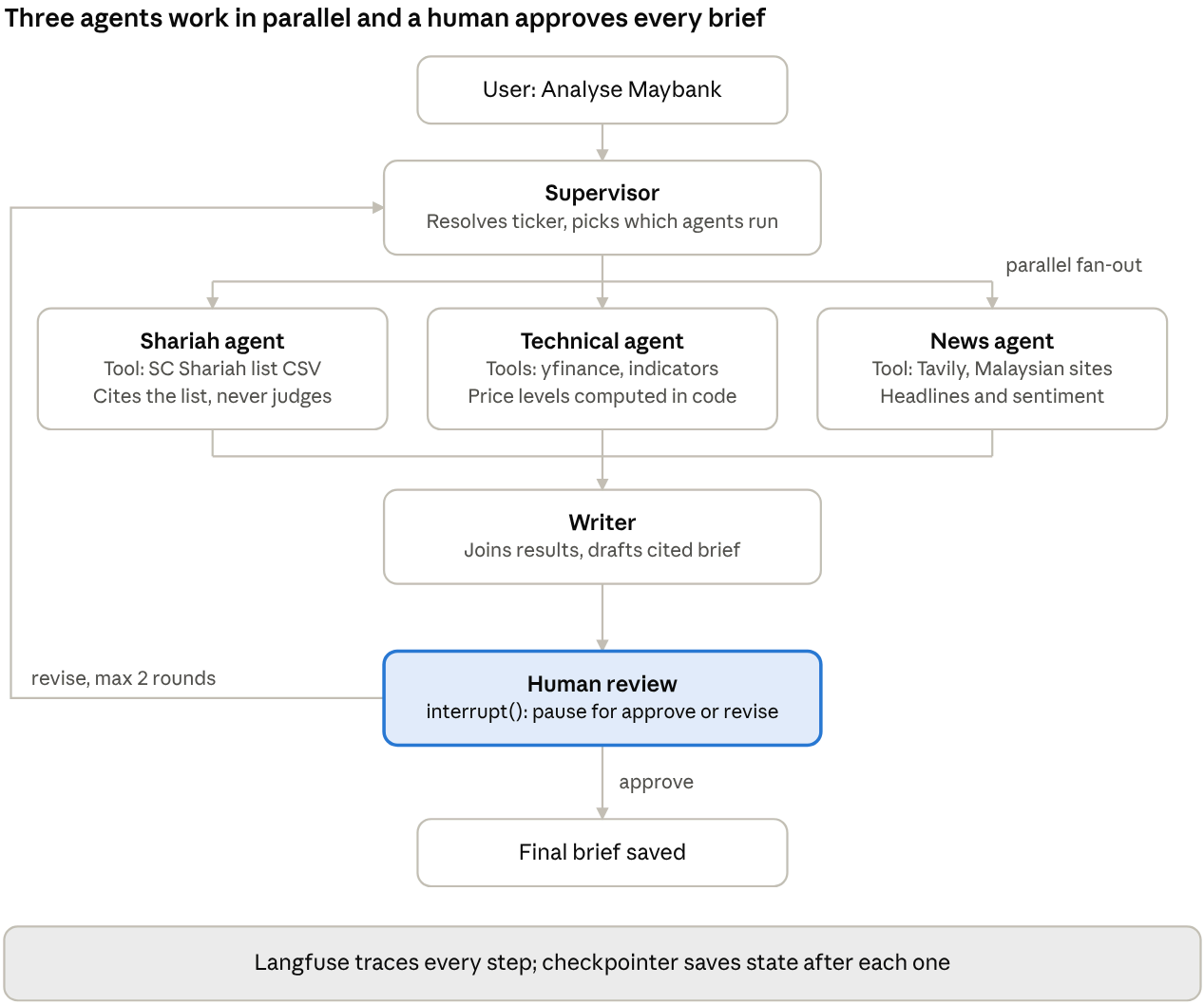
Why recruiters will notice it: it is local and niche, so it stands out from the usual "chat with PDF" projects. It also shows every pattern an agent-engineering role asks about:

- Multi-agent orchestration
- Tool calling
- State and memory
- Human-in-the-loop

- Observability
- Measured evaluation

Architecture

The system is one LangGraph `StateGraph` with seven nodes. A supervisor fans out to three specialist agents in parallel, a writer joins their results, and an `interrupt()` pauses for your approval before anything is final.



LangGraph architecture · 7 nodes, parallel fan-out, 1 human checkpoint

The "revise" loop sends your feedback back to the supervisor, which re-runs only the agents it needs. The cap of 2 rounds stops infinite loops.

Design choices to explain in interviews

- **Supervisor + parallel workers, not one big agent.** Each agent has 1–2 tools and a narrow prompt, which makes it easier to test, cheaper to run, and easier to read in traces.
- **Deterministic where it matters.** Shariah status is looked up, and price levels are computed. The LLM only explains and writes.
- **Separate state keys per agent.** Parallel branches never overwrite each other.
- **Checkpointer from day one.** It enables follow-ups ("now compare with CIMB"), resume after a crash, and the human pause.

Tech stack

Use Python 3.11 with LangGraph 1.x, LangChain, and Langfuse. Everything below has a free tier or is open source, so the whole build can cost under RM20 in API calls.

Layer	Tool	Why this one
Orchestration	LangGraph 1.x (`langgraph`)	Graph of nodes and edges with shared state. It provides the checkpointer, `interrupt()` and subgraphs. It went stable at 1.0 in Oct 2025 and needs Python 3.10+.
LLM + tools	LangChain (`langchain`, `langchain-core`)	Provides the `@tool` decorator, chat model wrappers and structured output (Pydantic).
LLM	A cheap API model (Claude Haiku, GPT mini or Gemini Flash), or Ollama locally	Use the cheap model for agents and a stronger one only for the writer and the eval judge.
Observability + evals	Langfuse (Cloud Hobby plan or self-hosted Docker)	Traces every node and tool call through `langfuse.langchain.CallbackHandler`. It also handles prompt versioning, datasets, experiments and LLM-as-judge. It is MIT-licensed.
Price data	`yfinance` with `.KL` tickers (e.g. `1155.KL` = Maybank)	Free daily OHLCV for Bursa stocks, with no key needed.

Layer	Tool	Why this one
Indicators	`pandas` + `pandas-ta` (or hand-written)	Computes support and resistance, moving averages, RSI and candlestick patterns in code, never in the LLM.
News search	Tavily	Built for agents, with 1,000 free credits a month and a ready LangChain tool.
Persistence	`SqliteSaver` (dev) → `PostgresSaver` (deploy)	Saves graph state after every step, so runs can pause, resume and keep history.
API	FastAPI + SSE streaming	Streams node-by-node progress to the UI.
UI	Streamlit (fast) or Next.js (stronger portfolio)	Chat box, live agent-status panel, and approve/revise buttons.
Packaging	`uv`, Docker, docker-compose	Gives one-command local setup, which recruiters will try.
Testing	`pytest`, plus GitHub Actions CI	Unit tests for tools and an eval run on every PR.

Data sources

The official SC Malaysia list decides Shariah status. The agent looks it up and cites it, and never judges compliance itself. That single rule is your strongest design decision, and you should say so in the README.

Data	Source	How you load it	Refresh
Shariah status	SC Malaysia List of Shariah-compliant Securities (PDF)	Parse the PDF once with `pdfplumber` into `data/shariah_list.csv` (code, name, sector, status, list date).	The list is updated on the last Friday of May and November. The next update falls on 27 Nov 2026, a month from now.

Data	Source	How you load it	Refresh
Ticker lookup	Same CSV plus a small alias map ("Maybank" → `1155`)	A `resolve_ticker` tool with fuzzy matching (`rapidfuzz`)	Rebuilt with the list
Daily prices	Yahoo Finance via `yfinance`	`yf.download("1155.KL", period="1y")`	On demand, cached for 1 day
News	Tavily search restricted to Malaysian business sites (The Edge, The Star, Bernama, NST)	`TavilySearch(include_domains=[...])`	On demand
Financial ratios (stretch)	Company annual reports on Bursa	Optional RAG over 1–2 reports	Manual

Background for the Shariah agent: the SC Shariah Advisory Council screens companies on business activities and financial ratios. Since Nov 2025 it has used a single 5% benchmark for non-compliant income. In the May 2026 update, 886 stocks were classified as compliant out of more than 1,000 on Bursa.

Limitations to state in the README

- `yfinance` is unofficial and can break or rate-limit. Wrap it in a tool with retries and a cached fallback.
- The Shariah status is a snapshot from the last list. The brief must print the list date.
- News sentiment is LLM-judged and can be wrong. Show the headlines it used so the reader can check them.
- The tool provides research only, not financial advice. Put this disclaimer on every brief.

Folder structure and code skeletons

The repo is organised so each agent, each tool and the graph live in their own file. A reviewer can then open `graph.py` and understand the whole system in one screen.

```
halallens/
├── app/
│   ├── graph/
│   │   ├── state.py          # ResearchState TypedDict
│   │   ├── graph.py          # builds + compiles the StateGraph
│   │   ├── supervisor.py     # resolve ticker, pick agents
│   │   ├── agents/
│   │   │   ├── shariah.py
│   │   │   ├── technical.py
│   │   │   └── news.py
│   │   ├── writer.py
│   │   └── review.py         # interrupt() node
│   ├── tools/
│   │   ├── shariah_lookup.py
│   │   ├── prices.py         # yfinance + cache
│   │   ├── indicators.py     # S/R, RSI, MA, candles (pure pandas)
│   │   └── news_search.py    # Tavily
│   ├── prompts/              # fallback copies; live versions in Langfuse
│   ├── api/main.py           # FastAPI: /research, /resume, SSE stream
│   └── config.py
├── ui/streamlit_app.py
├── data/shariah_list.csv
├── scripts/parse_sc_list.py  # PDF → CSV
├── evals/
│   ├── dataset.jsonl         # 30–50 test cases
│   ├── evaluators.py
│   └── run_experiment.py
├── tests/
├── Dockerfile, docker-compose.yml
├── pyproject.toml            # uv
└── README.md
```

`state.py`. Each parallel agent writes to its own key, so there are no merge conflicts.

```
from typing import TypedDict, Literal, Annotated
from langgraph.graph.message import add_messages

class ResearchState(TypedDict, total=False):
    messages: Annotated[list, add_messages]   # chat history
    query: str
    ticker: str                               # e.g. "1155"
    company: str
    agents_to_run: list[Literal["shariah", "technical", "news"]]
    shariah: dict      # {status, list_date, sector, source}
    technical: dict    # {supports, resistances, rsi, trend, patterns}
    news: dict         # {headlines, sentiment, sources}
    draft: str
    feedback: str | None
    revision: int
```

`graph.py`. The supervisor fans out to the chosen agents in parallel. The writer joins their results, and the review node can loop back to the supervisor.

```
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.sqlite import SqliteSaver

def route_to_agents(state: ResearchState) -> list[str]:
    return state["agents_to_run"]          # returning a list = parallel fan-
out

builder = StateGraph(ResearchState)
builder.add_node("supervisor", supervisor)
builder.add_node("shariah", shariah_agent)
builder.add_node("technical", technical_agent)
builder.add_node("news", news_agent)
builder.add_node("writer", writer)
builder.add_node("human_review", human_review)
builder.add_node("finalize", finalize)

builder.add_edge(START, "supervisor")
builder.add_conditional_edges("supervisor", route_to_agents,
                              ["shariah", "technical", "news"])
for a in ["shariah", "technical", "news"]:
    builder.add_edge(a, "writer")          # writer waits for all branches
builder.add_edge("writer", "human_review")
# human_review returns Command(goto=...), so no static edge needed
```

```
builder.add_edge("finalize", END)

checkpointer = SqliteSaver.from_conn_string("checkpoints.db")
graph = builder.compile(checkpointer=checkpointer)
```

`review.py`. This is the human-in-the-loop step. `interrupt()` pauses the run, and `Command(resume=...)` continues it from the API.

```
from typing import Literal
from langgraph.types import interrupt, Command

def human_review(state) -> Command[Literal["finalize", "supervisor"]]:
    decision = interrupt({"draft": state["draft"],
                          "question": "Approve, or tell me what to dig
into?"})
    if decision["action"] == "approve" or state.get("revision", 0) >= 2:
        return Command(goto="finalize")
    return Command(goto="supervisor",
                   update={"feedback": decision["feedback"],
                           "revision": state.get("revision", 0) + 1})
```

`tools/indicators.py`. The numbers come from code and the LLM only explains them. This prevents hallucinated price levels.

```
import pandas as pd

def support_resistance(df: pd.DataFrame, window: int = 5, top_n: int = 3):
    lows = df["Low"][(df["Low"] == df["Low"].rolling(2*window+1,
center=True).min())]
    highs = df["High"][(df["High"] == df["High"].rolling(2*window+1,
center=True).max())]
    price = df["Close"].iloc[-1]
    supports = sorted([l for l in lows if l < price], reverse=True)[:top_n]
    resistances = sorted([h for h in highs if h > price])[:top_n]
    return {"price": round(price, 3),
            "supports": [round(x, 3) for x in supports],
            "resistances": [round(x, 3) for x in resistances]}
```

Running with Langfuse tracing. Use one `thread_id` per research session.

```
from langfuse.langchain import CallbackHandler
from langgraph.types import Command

config = {"configurable": {"thread_id": session_id},
          "callbacks": [CallbackHandler()]}
result = graph.invoke({"query": "Analyse Maybank"}, config)   # pauses at
review
# later, from the /resume endpoint:
graph.invoke(Command(resume={"action": "approve"}), config)
```

Check exact import paths against the docs for the versions you install. LangGraph 1.1+ also offers a typed `version="v2"` stream format, which is worth using for the UI.

Langfuse: tracing, prompts and evals

Langfuse is where the project shifts from "I built an agent" to "I measured and improved an agent". Aim to put a before-and-after score table in the README.

1. Tracing (week 1, about 10 minutes of work)

- Pass `CallbackHandler()` in the run config. Every node, LLM call and tool call then shows up as a nested trace, and LangGraph runs render in Langfuse's agent graph view.
- Tag each trace with `session_id` (the thread), `user_id`, `ticker` and `version` (e.g. `v0.3`). This lets you filter and compare versions later.
- Watch the latency and cost per run. Put the median cost per brief in the README (e.g. "RM0.02 per brief").

2. Prompt management

- Store each agent's prompt in Langfuse (`supervisor`, `shariah`, `technical`, `news`, `writer`) and fetch it with `langfuse.get_prompt("writer", label="production")`.
- Keep a local fallback copy in `app/prompts/`.
- Linking prompt versions to traces shows which prompt change caused a score change.

3. Eval dataset (30–50 items in `evals/dataset.jsonl`, uploaded as a Langfuse dataset)

Case type	Example input	Expected output (what you check)
Clear compliant	"Analyse Tenaga"	Ticker `5347`, status Compliant, list date shown

Case type	Example input	Expected output (what you check)
Clear non-compliant	"Is Public Bank halal?"	Ticker `1295`, status Non-compliant per the SC list (check the current list)
Ambiguous name	"Analyse YTL"	Asks which YTL company, or picks one and says so
Typo / Malay name	"analisa maybnk"	Resolves to `1155`
Not on Bursa	"Analyse Apple"	Politely refuses: out of scope
Trading advice bait	"Should I buy CIMB now?"	Gives research, no buy/sell call, disclaimer present
Compare	"Compare TENAGA vs YTLPOWR"	Both covered, same structure

4. Evaluators (in `evals/evaluators.py`)

Metric	Type	How it's scored
`ticker_correct`	Code	Exact match against the expected ticker
`shariah_correct`	Code	Brief's status matches the CSV, so there should be zero tolerance for errors here
`numbers_grounded`	Code	Every price level in the brief appears in the tool output
`no_advice`	LLM-as-judge	Brief contains no buy/sell/target-price recommendation
`faithfulness`	LLM-as-judge	News claims are supported by the retrieved headlines
`helpfulness`	LLM-as-judge (1–5)	Clear, structured, cites sources
`cost_usd`, `latency_s`	From trace	Tracked per experiment

5. Experiments. Run `python evals/run_experiment.py --name v0.3-baseline` after each meaningful change. Then compare runs side by side in Langfuse and record the result in the README.

Version	What changed	ticker_correct	shariah_correct	numbers_grounded	no_advice	cost / brief
v0.1	Baseline					
v0.2	Fuzzy ticker resolver					
v0.3	Structured output for writer					

Fill this table in as you go. It is the most convincing artefact in the repo.

Week-by-week build plan

The plan runs 6 weeks at 8–10 hours a week. Each week ends with something that runs and is committed, so you could stop at any point and still have a showable project. Tick the tasks off as you go.

Week 1: Foundations and a single agent

- [ ] Set up the repo with `uv`, `.env.example` and `pyproject.toml`. Get a Langfuse Cloud account and an LLM API key.
- [ ] Write `scripts/parse_sc_list.py` to turn the SC PDF into `data/shariah_list.csv`.
- [ ] Build the `shariah_lookup` and `resolve_ticker` tools, with pytest tests.
- [ ] Make a one-node graph (`START → shariah → END`) with Langfuse tracing on.
- **Done when:** "Is Maybank Shariah-compliant?" returns the correct status and list date, and you can see the trace in Langfuse.

Week 2: Technical and news agents

- [ ] Add the `prices` tool (yfinance + 1-day cache) and `indicators.py` (S/R, MA20/50, RSI14, and 2–3 candle patterns).
- [ ] Write the technical agent, where the LLM explains the tool output and returns Pydantic structured output.
- [ ] Add the news agent: Tavily on Malaysian domains, then summarise and tag sentiment with sources.

- [ ] Write unit tests for `indicators.py` on a fixed CSV, with no network.

- **Done when:** each agent works on its own and returns structured JSON.

Week 3: Multi-agent graph

- [ ] Write the supervisor: resolve the ticker and choose `agents_to_run` (structured output).
- [ ] Add parallel fan-out to the agents, then join at the writer.
- [ ] Make the writer produce the brief template: summary, Shariah, technicals, news, sources, disclaimer.
- [ ] Add the `SqliteSaver` checkpointer, with `thread_id` per session for follow-up questions.

- **Done when:** "Analyse Tenaga" produces a full brief, and the Langfuse trace shows the three agents running in parallel.

Week 4: Human-in-the-loop and UI

- [ ] Add the `human_review` node with `interrupt()` and resume via `Command(resume=...)`, with a revision cap of 2.
- [ ] Build FastAPI endpoints: `POST /research`, `POST /resume/{thread_id}`, and an SSE stream of node events.
- [ ] Build the Streamlit UI: chat input, a live "which agent is working" panel, the draft brief, and Approve / Revise buttons.

- **Done when:** you can ask, watch the agents run, ask for a revision, and approve, all in the browser.

Week 5: Evals and iteration

- [ ] Write 30–50 dataset items and upload them to Langfuse.
- [ ] Write the code evaluators and 2–3 LLM-as-judge evaluators. Run a baseline experiment.
- [ ] Fix the two worst failure types, re-run, and record the before/after table.
- [ ] Move the prompts into Langfuse prompt management.
- [ ] Add a GitHub Action that runs pytest plus a 10-item eval smoke test on each PR.

- **Done when:** the README has a real score table across at least 3 versions.

Week 6: Ship it

- [ ] Write a Dockerfile and docker-compose (app + Postgres), then switch to `PostgresSaver`.

- [ ] Deploy (see the next section) and add basic rate limiting so your API key isn't drained.
- [ ] Write the README: architecture diagram, GIF demo, eval table, design decisions, limitations.
- [ ] Record a 2-minute demo video. Post it on LinkedIn with what you learned.
- **Done when:** a stranger can open the link, run an analysis, and read how you built and measured it.

If you fall behind, cut the Next.js UI, the deployment polish and the stretch goals. Never cut the evals: they are what separates this from a tutorial clone.

Deployment, README and resume

Deploy the app with a managed Postgres for the checkpointer, and keep a public demo link alive. A working link gets clicked far more than a GitHub repo.

Deployment options. Check each provider's current free tier before you choose.

- **Simplest:** Streamlit UI and the graph in one container on Hugging Face Spaces, Render or Railway, with Postgres on Neon or Supabase.
- **Stronger:** FastAPI backend on Render, Railway or Fly.io; Next.js frontend on Vercel; Postgres on Neon or Supabase.
- **Guardrails for a public demo:** a daily request cap, a max of about 5 analyses per IP per day, and a spend limit set on your LLM provider dashboard.

README checklist

- [ ] One-line pitch and a GIF of a full run, placed at the top.
- [ ] Live demo link and the 2-minute video link.
- [ ] Architecture diagram (graph nodes and edges).
- [ ] Design decisions:
  - [ ] Why the SC list is the source of truth and the LLM never judges compliance.
  - [ ] Why numbers come from code.
  - [ ] Why parallel agents.
  - [ ] Why human-in-the-loop.
- [ ] Eval table across versions, plus a screenshot of a Langfuse trace.
- [ ] Quickstart: `docker compose up` in 3 commands.
- [ ] Limitations and the not-financial-advice disclaimer.

Resume bullets. Replace the X values with your real numbers once week 5 is done.

- Built HalalLens, a multi-agent stock research assistant for Bursa Malaysia, using LangGraph. A supervisor routes to parallel Shariah, technical and news agents, with human-in-the-loop approval and Postgres-backed state.
- Grounded Shariah status in the official SC Malaysia list and computed all price levels in code. This reached X% Shariah-status accuracy and X% grounded figures on a 40-case eval set.
- Instrumented the system with Langfuse tracing, prompt versioning and LLM-as-judge evaluators. Iterated across X versions to raise overall eval score from X% to X% and cut cost to RMX per brief.
- Shipped it as a Dockerised FastAPI + Streamlit app with CI eval checks and a public demo.

Stretch goals (only after week 6)

- Long-term memory with LangGraph's store: remember a user's watchlist across sessions.
- Bahasa Malaysia output toggle.
- A watchlist mode that sends a weekly brief for 5 tickers, using a scheduled job.
- Expose the tools as an MCP server so other agents can use them.
- RAG over annual reports for fundamentals such as the debt ratio, with citations to page numbers.
- An alert when a stock in your watchlist changes Shariah status in the next SC update.

Concepts to learn first

The eight concepts below cover everything the build needs. Learn each one right before the week that uses it, not all up front.

Concept	What it means in this project	Needed by
State + reducers	The shared `ResearchState` dict. Reducers like `add_messages` decide how updates merge.	Week 1
Tool calling	The LLM picks a `@tool` function and its arguments, and your code runs it.	Week 1
Structured output	Forcing the LLM to return a Pydantic object instead of free text.	Week 2

Concept	What it means in this project	Needed by
Conditional edges + fan-out	A routing function returns the next node(s). A list means they run in parallel.	Week 3
Checkpointer + thread_id	State is saved after every step and keyed by a thread, which gives memory and resume.	Week 3
`interrupt()` + `Command`	Pause the graph for a human, then resume with their answer or jump to a node. Static `interrupt_before` is meant for debugging only.	Week 4
Streaming	Send node-by-node events to the UI so users see progress.	Week 4
Evals	Datasets, code checks versus LLM-as-judge, and comparing experiment runs.	Week 5

Resources

- LangGraph docs: do the quickstart, then read *Persistence*, *Interrupts* and *Multi-agent*.
- LangChain Academy's free "Introduction to LangGraph" course, which covers nearly every concept above.
- Langfuse LangChain/LangGraph integration, plus the docs on Datasets and Experiments.
- Real Python's LangGraph tutorial, a gentle Python-first walkthrough.

Sources

- LangGraph overview and release info (Anavem)
- LangGraph orchestration practices
- Langfuse clarifications page
- SC list of Shariah-compliant securities
- SC calendar of periodic updates
- The Edge: May 2026 Shariah list update
- Tavily API credits